

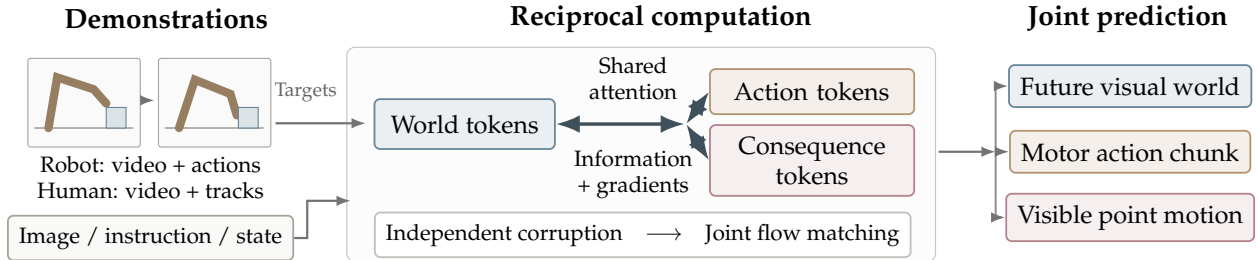
# JAM: Action-Grounded World Pretraining

Lixuan He

September 2026

## Abstract

Video pretraining provides a prior over how scenes evolve. Learning to act also requires a representation of how control relates to that evolution. We study whether action supervision can shape a pretrained world representation through the computation used to generate future observations. JAM, Joint Action-World Modeling, jointly generates future video, motor actions, and visual action consequences. A video backbone and an agent transformer exchange attention keys and values at every layer, allowing action prediction to update the world stream. Visual consequences describe image-space displacement and observability, extending the same training objective to human demonstrations with tracked motion. Independent corruption levels support learning from modalities with different levels of uncertainty. The available training record shows decreasing world, action, and consequence objectives in a completed task adaptation run, while foundation pretraining continues on robot and human data. Controlled comparisons are organized around the effects of information exchange, gradient routing, and consequence supervision on representation quality and transfer. This formulation makes the influence of action learning on world pretraining a concrete, testable property of the model.



**Figure 1: Learning an interaction through its commands and visible evolution.** Robot commands and available tracked motion supervise generated variables within a shared computation. Reciprocal attention carries their learning signals into the world representation. World, action, and consequence predictions describe the same future interaction.

## 1 Introduction

Video models learn regularities in appearance, motion, and scene evolution from visual sequences. These regularities provide useful starting points for embodied learning: a robot must anticipate how objects move, how contact changes a scene, and which futures are consistent with its observations. Recent work transfers video generation to control by extracting actions from imagined trajectories or adapting video models to generate actions directly (Du et al., 2023; Kim et al., 2026). This progress motivates a question about the pretraining process itself: *how should learning to act influence the representation used to predict the world?*

A manipulation video records an interaction through its visible outcome. Robot demonstrations additionally record the commands that accompanied it. These two descriptions constrain each other. The same scene can evolve differently under different commands, and a desired change in the scene can admit several feasible actions. A representation for embodied interaction should preserve these dependencies. Action grounding, in this work, refers to the extent to which a world representation retains information useful for relating commands to their visual consequences. We examine this property through prediction, controlled changes to the inputs, and transfer to new conditions.

We introduce JAM, Joint Action–World Modeling, as a way to expose the world representation to action supervision during pretraining. JAM generates actions and future observations within a shared denoising computation. A pretrained video stream and an agent stream exchange information throughout their depth. Their attention connections also carry gradients, so learning to predict actions can update the features used for world prediction. The central hypothesis is that this reciprocal computation encourages a representation of interaction that remains useful when the environment or task changes. The model makes this hypothesis experimentally accessible through separate controls over information visibility and gradient flow.

The relationship between commands and visible change also offers a route to human video. A motor command is specific to an embodiment and its controller. A visual action consequence describes where tracked points move in the image and whether they remain observable. JAM represents these consequences as generated tokens within the agent stream. Robot and human demonstrations can therefore supervise a common motion interface while retaining their own available annotations. This construction also provides a diagnostic: consequence prediction tests which aspects of interaction are accessible from the learned world representation.

Our study follows the influence of action supervision from the training computation to its downstream use. We first formulate the joint prediction problem and explain how the streams interact. We then describe pretraining with heterogeneous supervision and adaptation for control. The experiments connect optimization evidence to controlled tests of coupling, data composition, and transfer. Together, this organization asks when learning actions and visual consequences changes what a world model represents, and how that change can be measured.

## 2 Related Work

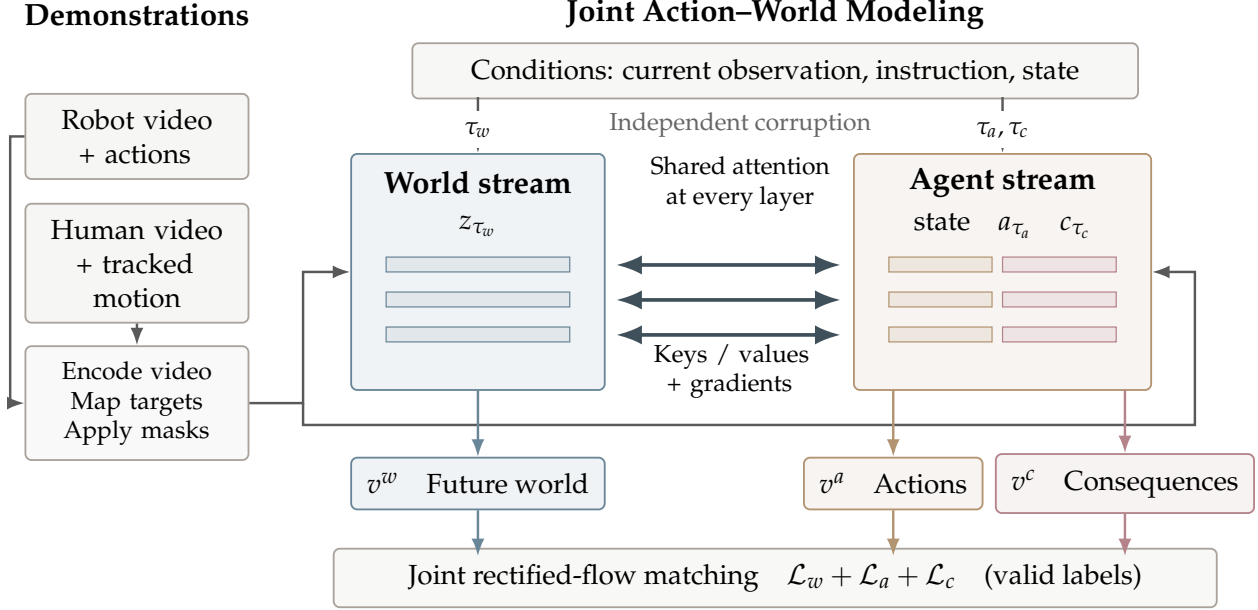
**Video priors for embodied learning.** Video generation learns temporal structure from visual data (Wan Team, 2025). UniPi uses generated videos as plans from which actions are recovered (Du et al., 2023). Cosmos Policy incorporates actions, future observations, and values into a pretrained video model through latent frames (Kim et al., 2026). These approaches establish practical routes from generative priors to control. JAM focuses on how action learning influences the video representation during embodied pretraining, with explicit controls over the connections carrying information and gradients.

**Joint action and world prediction.** Unified World Models (UWM) jointly diffuse actions and observations, sample their noise levels independently, and obtain policies and dynamics models through conditioning (Zhu et al., 2025). JAM builds on this joint-distribution perspective and uses rectified-flow training (Lipman et al., 2023; Liu et al., 2023b). Its study centers on a pretrained video stream coupled throughout its depth to an agent stream. Separating visibility from gradient routing permits comparisons between reading world features and updating those features through action supervision. Conditional sampling provides an additional use of the learned distribution; the representation question motivates the experimental design.

**Action representations across embodiments.** Robot policy pretraining aligns observations and language with commands across datasets and platforms (Kim et al., 2024; Khazatsky et al., 2024; Walke et al., 2023). Visual trajectories offer another interface to interaction: Any-point Trajectory Modeling learns motion representations for policies, and Track2Act converts predicted point tracks into executable motion (Wen et al., 2024; Bharadhwaj et al., 2024). EgoDex supplies human manipulation video with tracked hand poses (Hoque et al., 2025). JAM incorporates image-space consequences into its joint generative objective, alongside motor commands where available. The corresponding experiments examine command–consequence complementarity and transfer across embodiments under explicitly matched observation and annotation conditions.

## 3 Joint Action–World Modeling

The design starts from a simple requirement: action prediction and world prediction should be able to use and update each other’s representations. A joint objective specifies which variables are generated. The attention structure determines how learning those variables interacts.



**Figure 2: How action learning enters world pretraining.** Source adapters encode video and map available commands or tracked motion into masked targets. Independently corrupted future latents, actions, and consequences enter coupled world and agent streams. Attention exchanges carry information and gradients throughout the model. Separate velocity readouts receive their corresponding masked targets within the joint loss. World and agent streams share clean observation and language conditions; the state enters through the agent.

### 3.1 The prediction problem

Let  $o_0$  be the current observation,  $\ell$  a language instruction, and  $s$  the available proprioceptive state. We write  $h = (o_0, \ell, s)$  for these conditions, with a learned null token for missing state. Let  $z$  denote encoded future observations and  $a$  a chunk of future motor actions. JAM models  $p_\theta(z, a | h)$ . When visual consequence labels  $c$  are available, the same computation models  $p_\theta(z, a, c | h)$ . A fixed video autoencoder maps observations to latents; its decoder maps generated latents back to images. Action and consequence tokens preserve their respective control and image-space meanings.

For each generated modality  $m \in \{w, a, c\}$ , let  $x^w = z$ ,  $x^a = a$ , and  $x^c = c$ . We draw independent standard Gaussian noise  $\epsilon^m$  and a corruption level  $\tau_m$  between clean and fully noisy. Rectified-flow training constructs

$$x_{\tau_m}^m = (1 - \tau_m)x^m + \tau_m\epsilon^m, \quad u^m = \epsilon^m - x^m. \quad (1)$$

Here  $u^m$  is the target velocity along the interpolation from data to noise. The model predicts all available velocities from the corrupted modalities and the clean conditions:

$$(v_\theta^w, v_\theta^a, v_\theta^c) = F_\theta(z_{\tau_w}, a_{\tau_a}, c_{\tau_c}; h, \tau_w, \tau_a, \tau_c). \quad (2)$$

The current configuration samples consequence corruption independently of action corruption. This keeps the confidence of the consequence input distinct from that of the motor input. The formulation follows independent-modality corruption in UWM (Zhu et al., 2025); the coupling described below specifies how the pretrained representations participate.

### 3.2 Reciprocal computation across streams

The world stream retains a pretrained video transformer’s latent embeddings, blocks, and output projection. The agent stream contains a state token followed by action and consequence tokens. It uses separate input projections for each token type and noise-dependent adaptive normalization. Both streams receive the language condition. Their residual widths may differ; their attention projections share a common head geometry.

At each coupled layer, each stream forms its own queries, keys, and values. World queries attend to world and agent keys; agent queries attend to agent and world keys. In schematic form, for stream  $m$  and the other stream  $\bar{m}$ ,

$$H'_m = H_m + P_m \text{Attn}(Q_m, [K_m; K_{\bar{m}}], [V_m; V_{\bar{m}}]), \quad (3)$$

where  $H_m$  is the layer input,  $P_m$  maps attention outputs to its residual width, and brackets denote token concatenation. Each block then applies language cross-attention and a feed-forward update. Consequence tokens belong to the agent computation and participate in the same exchange. Figure 2 shows the complete training flow.

The default coupling is bidirectional at every layer, with the exchanged tensors attached to the computation graph. Thus action loss has a route into world parameters through the world keys and values read by the agent; world loss has a reciprocal route into agent parameters. These routes establish a mechanism for action supervision to influence the world representation. Their value for transfer is an empirical question. Visibility masks and gradient stops expose the mechanism to controlled comparison.

The current observation forms a clean prefix in the world sequence. Prefix queries attend only to the clean prefix, while generated world tokens can read the prefix and the agent stream. This preserves an observation context throughout joint prediction. The state token is a clean input to the agent stream and is excluded from the generation loss.

### 3.3 Learning under partial supervision

Each example carries coordinate masks  $M^m$  for the labels it supplies. We use a masked mean squared error,  $\text{MSE}_{M^m}$ , averaged over valid coordinates within that example. Let  $\mathcal{B}_m$  be the batch examples with at least one supervised coordinate for modality  $m$ . The implemented objective is

$$\mathcal{L} = \sum_{m \in \{w, a, c\}} \lambda_m \frac{1}{|\mathcal{B}_m|} \sum_{i \in \mathcal{B}_m} \omega(\tau_{m,i}) \text{MSE}_{M_i^m}(v_{\theta,i}^m, u_i^m), \quad (4)$$

where  $\lambda_m$  controls each modality's contribution and  $\omega$  weights its noise level. An empty supervised set contributes zero. World loss covers future latents; the clean prefix is excluded. Missing coordinates are masked after corruption as well as in the loss. Robot batches supervise actions and video, while human batches supervise video and the available consequences. Consequence annotations may also accompany robot data. Appendix A gives the implemented sampling distribution and normalization rules.

## 4 Action-Grounded World Pretraining

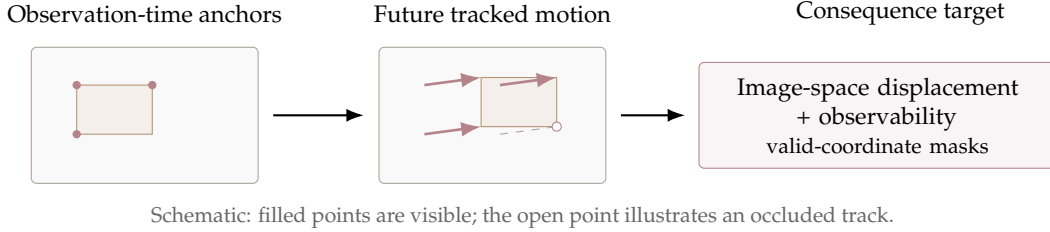
Joint prediction provides the learning mechanism. Its use across demonstrations depends on preserving what each source actually observes and controls. We therefore align the meaning of the inputs before combining their supervision.

### 4.1 Commands and their visual consequences

A source adapter maps motor commands into a typed layout for end-effector pose, gripper aperture, joints, and additional embodiment-specific coordinates. The adapter also records which coordinates exist. Source-specific normalization is applied after this mapping. Rotation coordinates retain their geometric convention, and gripper aperture follows a shared closed-to-open convention. This interface preserves physical meaning when sources populate different subsets of the layout.

Visual action consequences describe displacement from observation-time anchors. For keypoint  $k$  at future time  $t$ , let  $q_{t,k}$  be its image-normalized position,  $q_{0,k}$  its anchor, and  $b_{t,k}$  its observability. The target is

$$c_{t,k} = [g(q_{t,k} - q_{0,k}), 2b_{t,k} - 1], \quad (5)$$



**Figure 3: What consequence supervision represents.** Motion is expressed relative to an anchor in the current image. Visibility is predicted separately from displacement validity, so an occluded point supplies a visibility target while its displacement is masked. The active human adapter obtains tracks by projecting supplied hand poses; robot annotations can combine actuator tracks and scene queries. This is an explanatory drawing, not a model prediction.

where  $g$  is a fixed displacement gain. Tracked actuator points use their observation-time position as the anchor; scene probes use their query location in that frame. Masks distinguish missing slots from observed low visibility. The resulting sequence is projected into consequence tokens and trained by the same velocity objective as actions and video.

Figure 3 illustrates this interface. Point trackers provide one route to such labels (Karaev et al., 2024). The active human-data adapter instead projects supplied hand poses through the recorded camera calibration and combines their confidence with visibility (Hoque et al., 2025). These demonstrations supply tracked motion while motor commands remain unobserved. Consequences express visible motion in a common coordinate format; camera motion, projection, and embodiment still affect their distribution. Cross-embodiment invariance is therefore evaluated through held-out transfer.

## 4.2 Pretraining from heterogeneous demonstrations

Each training window contains an instruction, a current observation, future frames, and the available state, action, and consequence annotations. Camera views occupy fixed positions in an image canvas, which is encoded by the frozen video autoencoder. A frozen text encoder supplies language embeddings. Learned input projections route control and consequence coordinates into the agent stream. The pretrained world transformer and the new agent transformer are optimized together.

We sample sources using a temperature applied to their numbers of valid training windows. Each micro-batch contains a single source so its camera geometry and annotation pattern remain consistent. Robot and human batches enter the same optimization stage. Human data thus contributes a world objective and a consequence objective while robot data anchors the command interface. The current recipe and training budget appear in Section 5; larger acquired corpora enter the accounting when their caches are incorporated into a run.

Increasing data, updates, and model capacity answers different scaling questions. The study varies each separately: subsets of a fixed corpus test data diversity, checkpoint milestones test training duration, and agent capacity tests the cost of the learned control interface. The key comparison preserves robot exposure when adding human data, followed by a compute-matched comparison. This separates an additional supervision source from an additional optimization budget.

## 4.3 Adaptation and conditional use

Downstream adaptation retains the joint objective and replaces the data mixture with demonstrations for the target environment. At execution time, JAM starts the generated streams from noise, integrates predicted velocities toward clean samples, and executes a prefix of the action chunk before observing again. The world decoder and consequence readout make the jointly sampled future available for diagnostics.

The joint formulation also supports action-conditioned world prediction and inverse action prediction from future observations. Independent corruption varies uncertainty in each input. Each sampler specifies supplied variables, latent variables, and their integration schedules. The current policy uses a shared decreasing schedule; conditional comparisons await sampler validation.

## 5 Experiments

The evaluation connects reciprocal computation to control, transfer, and representation quality. Each comparison isolates one step in this proposed relationship.

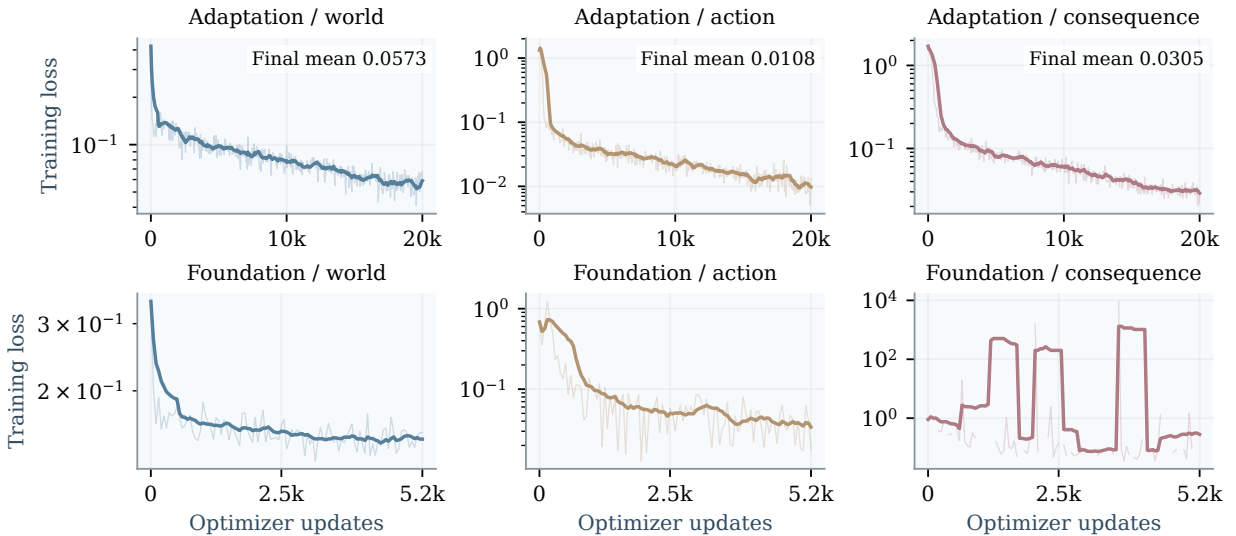
### 5.1 Experimental setup

JAM pretrains on DROID (Khazatsky et al., 2024), BridgeData V2 (Walke et al., 2023), and EgoDex (Hoque et al., 2025), combining robot commands with human motion supervision from the first update. The model pairs a pretrained video backbone with a newly initialized agent stream, totaling 6.02 billion parameters. Foundation training uses eight H200 accelerators and global batch 512. Episode-level holdouts separate training windows from open-loop evaluation. Downstream tests cover LIBERO (Liu et al., 2023a), RoboTwin 2.0 (Chen et al., 2025), VLABench (Zhang et al., 2025), and RoboCasa365 (Nasiriany et al., 2026); LIBERO-Plus (Fei et al., 2025) and LIBERO-PRO (Zhou et al., 2025) test distribution shifts.

Paired comparisons hold demonstrations, adaptation budget, samplers, and environment resets fixed. Control uses task success; representation tests use frozen-world probes, perceptual prediction error, and consequence displacement error. The planned protocol uses three training seeds with paired bootstrap intervals. The evidence snapshot contains a completed 20,000-update direct-adaptation run and foundation logs through update 5,200. These are measured optimization records. External benchmark scores retain source attribution; JAM benchmark cells marked T are layout targets awaiting evaluation. Appendix B specifies data proportions, optimization, evaluation manifests, and statistics.

### 5.2 Optimization and downstream control

Figure 4 records both stages of joint optimization, and Table 1 organizes downstream control by task family.



**Figure 4: Do the three objectives optimize together?** Measured world, action, and consequence losses during completed LIBERO adaptation (top) and the archived foundation prefix (bottom). Thin traces show logged batches; thick traces average the available supervised values within the last 11 records. Missing supervision is excluded, and each objective retains its own logarithmic scale. Top annotations summarize the final tenth of adaptation records. Foundation outliers remain visible. These curves measure training behavior; control and transfer require separate evaluation.

| A. LIBERO: task control |                   |                   |                   |                   |                   |
|-------------------------|-------------------|-------------------|-------------------|-------------------|-------------------|
| Model                   | Spatial           | Object            | Goal              | Long              | Mean              |
| OpenVLA                 | 84.7              | 88.4              | 79.2              | 53.7              | 76.5              |
| $\pi_0$                 | 98.0              | 96.8              | 94.4              | 88.4              | 94.4              |
| $\pi_{0.5}$             | 98.8              | 98.2              | 98.0              | 92.4              | 96.9              |
| OpenVLA-OFT             | 97.6              | 98.4              | 97.9              | 94.5              | 97.1              |
| X-VLA                   | 98.2              | 98.6              | 97.8              | 97.6              | 98.1              |
| R1                      | 99.6              | 99.6              | 99.8              | 98.2              | 99.3              |
| <b>JAM direct</b>       | 98.0 <sup>T</sup> | 98.4 <sup>T</sup> | 97.2 <sup>T</sup> | 95.2 <sup>T</sup> | 97.2 <sup>T</sup> |
| <b>JAM pretrained</b>   | 99.8 <sup>T</sup> | 99.8 <sup>T</sup> | 99.6 <sup>T</sup> | 98.4 <sup>T</sup> | 99.4 <sup>T</sup> |

| B. Transfer across task families |                   |                   |                   |                   |                   |
|----------------------------------|-------------------|-------------------|-------------------|-------------------|-------------------|
| Model                            | Twin: clean       | Twin: random      | Twin: mean        | VLABench          | Casa365           |
| $\pi_{0.5}$                      | 82.70             | 76.80             | 79.75             | 48.1              | 16.9              |
| R1                               | 93.74             | 93.46             | 93.60             | 58.9              | 38.2              |
| <b>JAM direct</b>                | 77.0 <sup>T</sup> | 72.0 <sup>T</sup> | 74.5 <sup>T</sup> | 24.8 <sup>T</sup> | 12.4 <sup>T</sup> |
| <b>JAM pretrained</b>            | 95.0 <sup>T</sup> | 94.2 <sup>T</sup> | 94.6 <sup>T</sup> | 59.8 <sup>T</sup> | 39.6 <sup>T</sup> |

**Table 1: Does joint pretraining improve downstream control?** Success percentages; Twin denotes RoboTwin 2.0 Full. External scores reproduce the [external reference R1](#) export. JAM pairs match adaptation settings. Casa365 compares external full-suite scores with JAM target-task scores, preserving their distinct scopes. **Superscript T marks layout targets awaiting measurement.** Pale blue identifies JAM rows.

### 5.3 Generalization under distribution shift

Table 2 reports visual, semantic, and embodiment shifts with benchmark-specific breakdowns.

| A. LIBERO-Plus: visual and interaction shifts |                   |                   |                   |                   |                   |                   |                   |                   |
|-----------------------------------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|
| Model                                         | Camera            | Robot             | Language          | Light             | Backgr.           | Noise             | Layout            | Mean              |
| $\pi_0$                                       | 13.8              | 6.0               | 58.8              | 85.0              | 81.4              | 79.0              | 68.9              | 53.6              |
| $\pi_{0.5}$                                   | 78.4              | 73.6              | 80.8              | 96.2              | 94.1              | 89.0              | 84.5              | 84.4              |
| OpenVLA-OFT                                   | 56.4              | 31.9              | 79.5              | 88.7              | 93.3              | 75.8              | 74.2              | 69.6              |
| Cosmos Policy                                 | 75.8              | 63.3              | 81.7              | 96.5              | 88.9              | 92.7              | 82.2              | 82.2              |
| R1                                            | 33.8              | 76.1              | 88.0              | 97.0              | 87.1              | 39.8              | 77.5              | 69.2              |
| <b>JAM direct</b>                             | 28.8 <sup>T</sup> | 36.4 <sup>T</sup> | 62.5 <sup>T</sup> | 71.9 <sup>T</sup> | 64.1 <sup>T</sup> | 45.3 <sup>T</sup> | 67.6 <sup>T</sup> | 53.8 <sup>T</sup> |
| <b>JAM pretrained</b>                         | 86.2 <sup>T</sup> | 87.1 <sup>T</sup> | 91.3 <sup>T</sup> | 95.4 <sup>T</sup> | 93.7 <sup>T</sup> | 86.9 <sup>T</sup> | 90.8 <sup>T</sup> | 90.2 <sup>T</sup> |

| B. VLABench: semantic and visual transfer |                   |                   |                   |                   |                   |                   |  |
|-------------------------------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|--|
| Model                                     | In-dist.          | Category          | Reasoning         | Instruction       | Texture           | Mean              |  |
| $\pi_0$                                   | 47.0              | 21.2              | 29.1              | 17.3              | 32.2              | 29.4              |  |
| $\pi_{0.5}$                               | 65.4              | 38.2              | 43.9              | 48.2              | 44.9              | 48.1              |  |
| R1                                        | 83.4              | 38.1              | 58.0              | 53.8              | 61.4              | 58.9              |  |
| <b>JAM direct</b>                         | 40.0 <sup>T</sup> | 16.0 <sup>T</sup> | 24.0 <sup>T</sup> | 22.0 <sup>T</sup> | 22.0 <sup>T</sup> | 24.8 <sup>T</sup> |  |
| <b>JAM pretrained</b>                     | 84.0 <sup>T</sup> | 39.0 <sup>T</sup> | 60.0 <sup>T</sup> | 54.0 <sup>T</sup> | 62.0 <sup>T</sup> | 59.8 <sup>T</sup> |  |

| C. Held-out conditions and embodiments |                   |                   |                   |                   |
|----------------------------------------|-------------------|-------------------|-------------------|-------------------|
| Model                                  | PRO: state        | PRO: task         | Twin: random      | Embodiment        |
| <b>JAM direct</b>                      | 26.0 <sup>T</sup> | 6.0 <sup>T</sup>  | 14.2 <sup>T</sup> | 28.0 <sup>T</sup> |
| <b>JAM pretrained</b>                  | 46.8 <sup>T</sup> | 12.6 <sup>T</sup> | 62.4 <sup>T</sup> | 42.0 <sup>T</sup> |

**Table 2: Which shifts preserve a pretraining gain?** Success percentages. External values retain source aggregation. Panel C pairs resets; Twin evaluates randomized conditions after clean-only adaptation. Embodiment transfer holds out the target platform from pretraining. Appendix B specifies evaluation and statistics. **Superscript T marks layout targets awaiting measurement.** Pale blue identifies JAM rows.



## 5.4 Reciprocal computation and representation

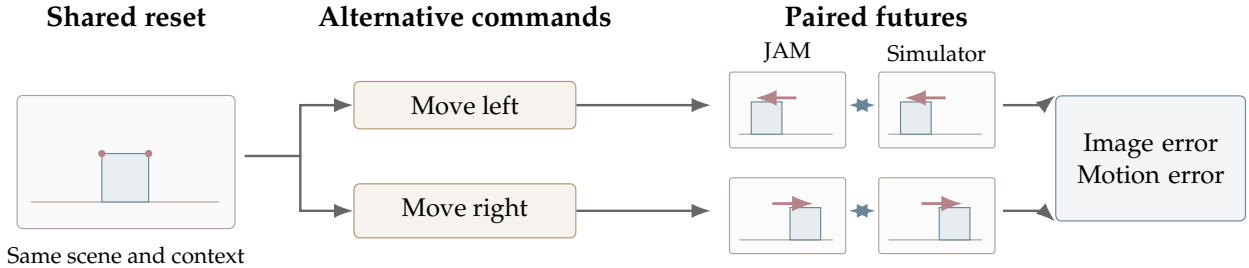
Table 3 links the coupling controls to representation diagnostics, with the paired intervention protocol illustrated in Figure 5.

| A. Information exchange and gradient routing |           |           |          |                   |                   |                   |                   |
|----------------------------------------------|-----------|-----------|----------|-------------------|-------------------|-------------------|-------------------|
| Variant                                      | W reads A | A reads W | Gradient | LIBERO            | Plus              | A probe           | C error           |
| World only                                   | Off       | Off       | Off      | 94.0 <sup>T</sup> | 42.0 <sup>T</sup> | 0.30 <sup>T</sup> | 0.18 <sup>T</sup> |
| Action only                                  | Off       | Off       | Off      | 95.0 <sup>T</sup> | 45.0 <sup>T</sup> | 0.32 <sup>T</sup> | 0.17 <sup>T</sup> |
| Independent                                  | Off       | Off       | Off      | 96.1 <sup>T</sup> | 47.6 <sup>T</sup> | 0.38 <sup>T</sup> | 0.15 <sup>T</sup> |
| Auxiliary                                    | Off       | On        | Off      | 96.8 <sup>T</sup> | 49.4 <sup>T</sup> | 0.40 <sup>T</sup> | 0.12 <sup>T</sup> |
| One-way                                      | Off       | On        | On       | 97.0 <sup>T</sup> | 51.7 <sup>T</sup> | 0.46 <sup>T</sup> | 0.10 <sup>T</sup> |
| <b>JAM joint</b>                             | On        | On        | On       | 97.2 <sup>T</sup> | 53.8 <sup>T</sup> | 0.52 <sup>T</sup> | 0.08 <sup>T</sup> |

| B. Representation and consequence diagnostics |             |            |             |                   |                   |
|-----------------------------------------------|-------------|------------|-------------|-------------------|-------------------|
| Diagnostic                                    | Metric      | Reference  | Test        | Ref. score        | Test score        |
| Action probe                                  | R-squared   | Auxiliary  | JAM         | 0.40 <sup>T</sup> | 0.52 <sup>T</sup> |
| Consequence                                   | ADE         | Auxiliary  | JAM         | 0.12 <sup>T</sup> | 0.08 <sup>T</sup> |
| Intervention                                  | ADE         | Auxiliary  | JAM         | 0.18 <sup>T</sup> | 0.10 <sup>T</sup> |
| World forecast                                | Image dist. | Action     | Action + C  | 0.24 <sup>T</sup> | 0.20 <sup>T</sup> |
| C conditioning                                | ADE         | Observed C | Predicted C | 0.10 <sup>T</sup> | 0.13 <sup>T</sup> |

**Table 3: What does reciprocal action–world learning change?** W, A, and C denote world, action, and consequence. Panel A fixes clean conditions; Gradient denotes action-loss access to world parameters; the probe reports R-squared and C error is average displacement error (ADE). Panel B pairs each diagnostic with its reference and test condition. Image distance measures perceptual prediction error. Observed and predicted consequences remain separate conditions, including a possible prediction gap. **Superscript T marks layout targets awaiting measurement.** Pale blue identifies JAM rows.



**Figure 5: Does a changed command produce the corresponding future?** Protocol schematic for the intervention study. Each command starts from the same saved simulator state; its predicted future is scored against the rollout produced by that command. The illustrated blocks and arrows explain pairing and are not generated rollouts or measured outcomes.



## 5.5 Data, capacity, and supervision

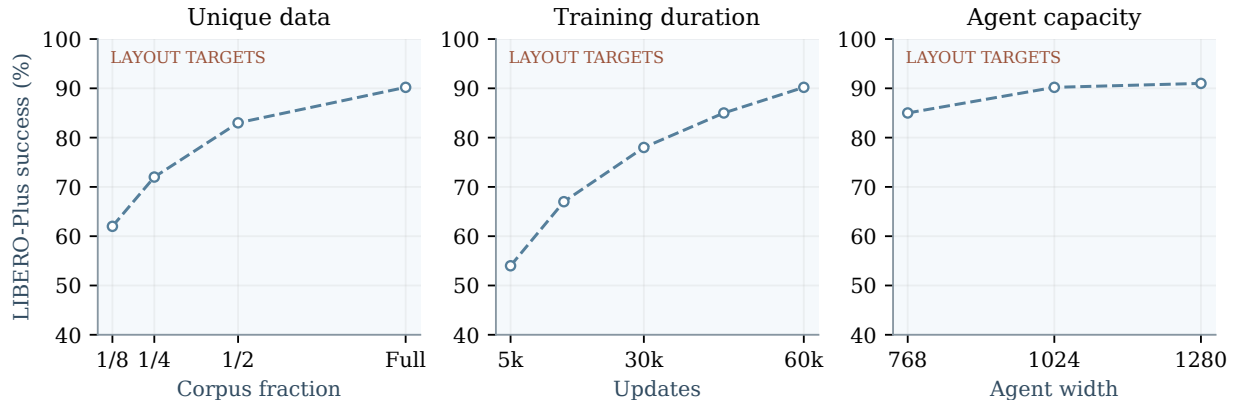
Table 4 and Figure 6 separate supervision source, unique data, optimization duration, and agent capacity.

| A. Which supervision source contributes? |              |       |         |                   |                   |                   |
|------------------------------------------|--------------|-------|---------|-------------------|-------------------|-------------------|
| Mixture                                  | Robot budget | Human | Human C | LIBERO            | Plus              | C error           |
| Robot                                    | Matched      | Off   | Off     | 98.4 <sup>T</sup> | 74.2 <sup>T</sup> | 0.12 <sup>T</sup> |
| + human video                            | Matched      | On    | Off     | 98.9 <sup>T</sup> | 82.6 <sup>T</sup> | 0.10 <sup>T</sup> |
| <b>JAM mixture</b>                       | Matched      | On    | On      | 99.4 <sup>T</sup> | 90.2 <sup>T</sup> | 0.08 <sup>T</sup> |

| B. Capacity and supervision density |         |        |         |                   |
|-------------------------------------|---------|--------|---------|-------------------|
| Axis                                | Setting | Corpus | Updates | Plus              |
| Agent width                         | 768     | Full   | 60k     | 85.0 <sup>T</sup> |
| Agent width                         | 1024    | Full   | 60k     | 90.2 <sup>T</sup> |
| Agent width                         | 1280    | Full   | 60k     | 91.0 <sup>T</sup> |
| Action labels                       | 25%     | Full   | 60k     | 82.0 <sup>T</sup> |
| Action labels                       | 50%     | Full   | 60k     | 86.0 <sup>T</sup> |
| Action labels                       | 100%    | Full   | 60k     | 90.2 <sup>T</sup> |

**Table 4: Which data and scale choices support action grounding?** Panel A matches robot exposure and varies human video and consequence supervision. Panel B fixes the video backbone, corpus, and update budget; blue rows identify the reference settings. Success is in percent and C error uses image-normalized ADE. Figure 6 separately varies unique data and update count. **Superscript T marks layout targets awaiting measurement.** Pale blue identifies JAM rows.



**Figure 6: How does the study separate data, training, and capacity?** Layout targets for LIBERO-Plus success. Each panel varies one axis while fixing the other two: corpus fraction at 60,000 updates, update count on the full corpus, and agent width at full corpus and 60,000 updates. All plotted points are PLANNED-VALUE, awaiting measurement. Dashed curves and their ordering specify the display, with no empirical scaling claim.

## 6 Conclusion

JAM places action prediction inside the computation used to learn world dynamics. Its coupled streams give action supervision a direct gradient route into a pretrained video representation, while visual consequences provide a motion target shared by robot and human demonstrations. The completed adaptation record shows that the implementation can optimize world, action, and consequence prediction together. Foundation training and the controlled evaluation address the remaining scientific question: when does this coupling produce a representation that transfers to new interactions? The present evidence supports the training formulation and motivates measuring representation change separately from optimization progress.

## References

- Homanga Bharadhwaj, Roozbeh Mottaghi, Abhinav Gupta, and Shubham Tulsiani. Track2Act: Predicting Point Tracks from Internet Videos enables Generalizable Robot Manipulation. In *European Conference on Computer Vision*, 2024. URL <https://arxiv.org/abs/2405.01527>.
- Tianxing Chen, Zanzin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, et al. RoboTwin 2.0: A Scalable Data Generator and Benchmark with Strong Domain Randomization for Robust Bimanual Robotic Manipulation. *arXiv preprint arXiv:2506.18088*, 2025. URL <https://arxiv.org/abs/2506.18088>.
- Yilun Du, Mengjiao Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Joshua B. Tenenbaum, Dale Schuurmans, and Pieter Abbeel. Learning Universal Policies via Text-Guided Video Generation. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2302.00111>.
- Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, et al. LIBERO-Plus: In-depth Robustness Analysis of Vision-Language-Action Models. *arXiv preprint arXiv:2510.13626*, 2025. URL <https://arxiv.org/abs/2510.13626>.
- Ryan Hoque, Peide Huang, David J. Yoon, Mouli Sivapurapu, and Jian Zhang. EgoDex: Learning Dexterous Manipulation from Large-Scale Egocentric Video. *arXiv preprint arXiv:2505.11709*, 2025. URL <https://arxiv.org/abs/2505.11709>.
- Nikita Karaev, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. Co-Tracker: It is Better to Track Together. In *European Conference on Computer Vision*, 2024. URL <https://arxiv.org/abs/2307.07635>.
- Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, et al. DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset. *arXiv preprint arXiv:2403.12945*, 2024. URL <https://arxiv.org/abs/2403.12945>.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, et al. OpenVLA: An Open-Source Vision-Language-Action Model. *arXiv preprint arXiv:2406.09246*, 2024. URL <https://arxiv.org/abs/2406.09246>.
- Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, and Jinwei Gu. Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2601.16163>.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow Matching for Generative Modeling. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.02747>.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. *arXiv preprint arXiv:2306.03310*, 2023a. URL <https://arxiv.org/abs/2306.03310>.
- Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow. In *International Conference on Learning Representations*, 2023b. URL <https://arxiv.org/abs/2209.03003>.
- Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. RoboCasa365: A Large-Scale Simulation Framework for Training and Benchmarking Generalist Robots. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2603.04356>.
- Homer Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Max Du, Chongyi Zheng, et al. BridgeData V2: A Dataset for Robot Learning at Scale. *arXiv preprint arXiv:2308.12952*, 2023. URL <https://arxiv.org/abs/2308.12952>.
- Wan Team. Wan: Open and Advanced Large-Scale Video Generative Models. *arXiv preprint arXiv:2503.20314*, 2025. URL <https://arxiv.org/abs/2503.20314>.
- Chuan Wen, Xingyu Lin, John So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point Trajectory Modeling for Policy Learning. In *Robotics: Science and Systems*, 2024. URL <https://arxiv.org/abs/2401.00025>.
- Shiduo Zhang, Zhe Xu, Peiju Liu, Xiaopeng Yu, Yuan Li, Qinghui Gao, Zhaoye Fei, Zhangyue Yin, Zuxuan Wu, Yungang Jiang, and Xipeng Qiu. VLABench: A Large-Scale Benchmark for Language-Conditioned Robotics Manipulation with Long-Horizon Reasoning Tasks. In *IEEE/CVF International Conference on Computer Vision*, 2025. URL <https://arxiv.org/abs/2412.18194>.
- Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. LIBERO-PRO: Towards Robust and Fair Evaluation of Vision-Language-Action Models Beyond Memorization. *arXiv preprint arXiv:2510.03827*, 2025. URL <https://arxiv.org/abs/2510.03827>.
- Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified World Models: Coupling Video and Action Diffusion for Pretraining on Large Robotic Datasets. *arXiv preprint arXiv:2504.02792*, 2025. URL <https://arxiv.org/abs/2504.02792>.

## Appendix

### A Implementation and training details

**Prediction horizons and coordinates.** The current model predicts 32 action steps in an 80-dimensional typed layout. Consequences use 16 steps with 48 available keypoint slots, each containing two displacement coordinates and one observability coordinate. The agent sequence therefore contains 49 tokens: one state token, 32 action tokens, and 16 consequence tokens. Slots are shared storage positions with source-specific validity masks. The current LIBERO adapter fills 39 consequence slots; EgoDex fills 14 hand-point slots. Unused slots carry a false mask.

Actions use source adapters that preserve the physical meaning of pose, aperture, and joint coordinates. Rotation is represented by the first two columns of its rotation matrix, concatenated in column order, and bypasses statistical scaling. Gripper aperture is first expressed from closed to open in the interval  $[0, 1]$  and then mapped to  $[-1, 1]$ . The active robot recipe uses per-source first and ninety-ninth percentiles for the remaining valid coordinates, with clipping to  $[-1, 1]$ . Min-max and standard-score normalization are supported alternatives; the archived runs identify the active choice. Displacement uses image-normalized coordinates with gain 4. Observability remains a separate target, including for occluded points whose displacement is masked.

**Temporal and visual inputs.** A window selects nine frames at a stride of four source steps, spanning 33 time indices. The video autoencoder produces a clean observation latent followed by future latents. The image canvas is  $384 \times 256$  pixels for the two-view template and  $384 \times 320$  for the three-view template. The template reserves positions for unavailable views rather than inventing additional observations. DROID, Bridge, and EgoDex retain native frame rates of 15, 5, and 30 Hz in the resolved recipe. Consequently, a step horizon corresponds to different elapsed durations across sources. Duration-matched comparisons are a separate control for cross-source transfer.

**Noise sampling and weighting.** For a uniform random variable  $u$  and shift  $r$ , the implemented level is  $\tau = ru / (1 + (r - 1)u)$ . The world and action shifts are 5 in the archived runs; the independent consequence draw uses the action shift. The bell-shaped weight is proportional to  $\exp[-2(\tau - 1/2)^2] - \exp(-1/2)$ , normalized by its mean under the shifted sampling distribution. World, action, and consequence levels are independent during training. The default sampling path decreases all three levels together with 10 Euler steps. Current training uses no additional probability mass on clamped boundary conditions. A boundary-mixture comparison is planned independently of the primary coupling comparison.

**Completed adaptation and active pretraining.** The completed LIBERO run uses 20,000 optimizer updates, batch 16 per device on four H200 accelerators, one accumulation step, and seed 0. World and agent learning rates both start at 0.0001 with 1,000 warmup updates and cosine decay to 0.01 of the peak. The foundation run uses the settings in Appendix B, with a warmup-stable-decay schedule ending at 0.05 of the peak. Training uses bfloat16 computation, float32 gradient reduction, and activation checkpointing. Foundation training uses hybrid sharding across two nodes of four accelerators. The video backbone is initialized from released video weights; embodied foundation training is performed within JAM.

The archived foundation configuration contains DROID, Bridge, and full EgoDex from its first optimizer update. An earlier robot-only stage pointer was replaced before training began. Checkpoints are saved every 300 updates. The first retained milestone release is at update 5,100, and the archived log extends to 5,200. The intended downstream launch point is the 15,000-update foundation milestone. These milestones describe training duration rather than completed scaling comparisons. The completed LIBERO adaptation has 401 logged points; its final-window summary averages the last 10% of those points. A zero consequence loss in a robot-only microbatch indicates absent supervision, so foundation summaries condition on the recorded supervised fraction.

### B Experimental setup and evaluation

**Data and splits.** The active pretraining mixture contains DROID (Khazatsky et al., 2024), BridgeData V2 (Walke et al., 2023), and EgoDex (Hoque et al., 2025); The active caches contain 1,674,648 DROID, 454,690

Bridge, and 738,550 EgoDex training windows. Sampling temperature 0.7 gives source probabilities of approximately 50.9%, 20.4%, and 28.7%. The cache reserves an episode-level holdout using a deterministic one-in-ten partition. All windows from an episode follow that partition. The active mixture contains the training portions of these sources and excludes the benchmark datasets below. Holdout windows serve open-loop checks; environment resets define the separate closed-loop test sets. Dataset expansion is tracked separately from data already consumed by training.

**Model and optimization.** The world stream uses Wan2.2-TI2V-5B through its released Diffusers implementation, initialized with public video weights. The agent stream is trained from scratch. The combined model has approximately 6.02 billion parameters, including a 1.02-billion-parameter agent with width 1,024 and 30 layers. Both streams use 24 attention heads of dimension 128. The video autoencoder and text encoder remain frozen. Foundation training uses eight H200 accelerators, a batch of 16 per device, and four accumulation steps, giving a global batch of 512. AdamW uses world/agent learning rates of 0.00005/0.0001, momentum coefficients 0.9/0.95, weight decay 0.01, and gradient clipping at 1. The planned budget is 60,000 updates with 1,500 warmup updates and a final 6,000-update decay. All three loss weights are 1. The resolved configuration uses independent corruption levels and zero explicit boundary-sampling probability. The completed direct-adaptation run uses four H200 accelerators, global batch 64, and a cosine learning-rate schedule; Appendix A records the remaining adaptation settings.

**Tasks and comparison protocol.** The core downstream arenas are LIBERO (Liu et al., 2023a), RoboTwin 2.0 Full (Chen et al., 2025), VLABench (Zhang et al., 2025), and the RoboCasa365 target-task split (Nasiriany et al., 2026). Direct adaptation starts from the public video initialization plus a fresh agent; pretrained adaptation starts from a JAM foundation release. The paired comparison fixes the demonstrations, action adapter, training budget, sampler, and environment version. LIBERO-Plus (Fei et al., 2025) and LIBERO-PRO (Zhou et al., 2025) supply robustness tests. RoboTwin clean-to-randomized transfer is evaluated separately from training on its Full mixture. The proposed final protocol uses three training seeds, 50 trials per LIBERO task, and the official one-trial-per-variant LIBERO-Plus enumeration. Each remaining suite follows its pinned task and reset manifest. Full-suite external averages remain distinct from the current task-subset experiments.

**Metrics and statistics.** Control is measured by task success rate, with subtask progress retained where supplied by the benchmark. World prediction uses perceptual image distance and tracked-motion error. Consequence prediction uses average displacement error in image-normalized coordinates and visibility calibration. Frozen-world probes predict actions and consequences from intermediate world features with generated action and consequence inputs fully corrupted. A shuffled-label probe and an observation-only predictor establish reference levels. Planned uncertainty estimates use a paired hierarchical bootstrap over training seeds, tasks, and shared episode resets with 10,000 resamples and 95% intervals. Open-loop metrics use episode-level resampling.

## C Controls required by the scientific question

**Separating information and gradients.** World-only pretraining supervises future video. Action-only pretraining supervises commands. Independent multitask pretraining supervises both with separate generated-stream computations. Auxiliary supervision reads detached world features. The attached one-way arm uses the same visibility while allowing action gradients through world keys and values. Reciprocal JAM additionally exposes agent keys and values to world queries. Each pretraining arm receives the same source sequence and then the same downstream adaptation procedure. Direct adaptation from public video weights supplies the separate baseline for the value of embodied pretraining.

Strict isolation of coupling requires identical clean conditions in every arm. In the audited implementation, disabling agent visibility also removes the state token from the world stream’s attention. The existing visibility switches therefore change both generated-token exchange and part of the conditioning interface. The planned comparison routes equivalent static state and current-observation information to every arm before interpreting a performance difference as an effect of coupling. The paper’s target table describes this stricter protocol, whose results remain awaiting measurement. It does not treat the existing switches as an already completed controlled experiment.

**Representation probes and interventions.** A world probe freezes the pretrained parameters and reads

the same selected layer and pooling rule in every arm. Generated action and consequence tokens are fully corrupted during feature extraction; their clean targets are used only to train or evaluate the probe on separate episodes. A linear probe tests accessible information, while a capacity-matched nonlinear probe checks dependence on readout capacity. Observation-only and shuffled-label predictors provide reference levels. World features are compared at fixed noise levels and matched visible context.

Action shuffling tests sensitivity to the action input. Counterfactual prediction additionally requires a target future obtained by executing the alternative action from the same simulator state. The intervention protocol therefore saves a reset state, executes each admissible command sequence, and scores each generated future against its matching rollout. Corruption tests vary timing and command magnitude within valid control limits. This distinguishes sensitivity from correct prediction of an intervention.

Consequence conditioning compares commands alone, observed consequences alone, and both together on the same test windows. Predicted consequences form a separate condition that retains their model error. The resulting gap measures the cost of prediction and any distribution shift at the conditioning interface. A held-out-embodiment study excludes the target platform from pretraining and fixes its downstream demonstration budget. Camera configuration, temporal duration, and available keypoint groups accompany each result.

**Conditional sampling checks.** Every conditional use specifies all generated variables. For action-conditioned world generation, an unobserved consequence stream remains latent and must be denoised on its own schedule. The audited action-clamped sampler couples its default consequence schedule to the action schedule; supplying only actions can therefore leave an initialized consequence latent unchanged. Conditional benchmark measurements require this behavior to be resolved or all clamped variables to be supplied explicitly. The manuscript consequently treats policy sampling as the primary implementation and conditional comparisons as pending validation. Independent training levels define the corruption family; each deployment sampler still requires its own correctness and fidelity check.

## D Evidence boundaries and failure analysis

The evidence package contains the completed adaptation log, its result summary, a fixed prefix of the ongoing foundation log, and sanitized resolved configurations. The public manifest records source paths, file hashes, the audit revision, and the status of each artifact. Local asset paths are replaced by logical references in the published configurations; original-file hashes preserve the connection to the inspected source. These configuration snapshots document the runs and are not standalone launch configurations.

The LIBERO closed-loop evaluation was interrupted before a complete suite result file was produced. Partial rollout videos therefore contribute neither success rates nor selected qualitative evidence to the paper. Foundation transfer, coupling ablations, and consequence probes await completed runs. External reference results retain their own provenance and evaluation scope. The planned tables and scaling figure use design values inherited or extended from the working manuscript; they are layout targets rather than preregistered outcomes or model predictions.

The data audit identified extreme displacement labels in a subset of human-video windows. A subsequently added displacement mask is configured to reject values beyond magnitude 8; the first archived training segment precedes activation of this correction. Its losses retain the original label distribution. Evaluating the filter requires a separately identified segment and comparison. This example motivates tracking label confidence and projection failure alongside prediction error. Camera motion, occlusion, and the mismatch between observed and predicted consequence distributions are additional failure settings in the evaluation design.

## E External reference results

Table 5 reorganizes source-reported results around the capabilities they evaluate. The linked source identifies the external method and retains its original table entries. R1 is a provenance identifier used to distinguish that system from JAM; the artifact includes an immutable source URL and an exact source pointer for every copied row.

| <b>R1: source protocol anchors</b> |         |            |         |             |         |
|------------------------------------|---------|------------|---------|-------------|---------|
| Benchmark                          | Success | Benchmark  | Success | Benchmark   | Success |
| LIBERO                             | 99.3    | Twin Full  | 93.60   | VLABench    | 58.9    |
| Casa365                            | 38.2    | Dojo       | 11.92   | EBench      | 49.4    |
| Plus                               | 69.2    | Twin clean | 89.4    | Twin random | 48.7    |
| Twin mean                          | 69.0    | Casa-GR1   | 60.5    | LIBERO Long | 98.2    |

**Table 5: External reference coverage.** Source-reported success percentages from [external reference R1](#). Dojo denotes RoboDojo and Casa-GR1 denotes RoboCasa-GR1. Twin clean, random, and mean refer to the clean-to-randomized protocol. Exact source strings and row pointers accompany every value.

## F Reproduction and experiment ledger

Measured logs, external references, and layout targets occupy separate artifact files. The README gives the complete build procedure. The commands `python3 scripts/build_artifacts.py` and `python3 scripts/validate.py` regenerate the numerical displays and check evidence consistency, respectively.

The protocol ledger links each scientific question to inspected source configurations and required result artifacts. It identifies controls that still require implementation changes. Promoting a target to a measurement requires its result file, checkpoint, evaluation manifest, and aggregation command; the corresponding scientific claim is then reviewed against that evidence.